

Object Oriented Programming (OOP)

[1] What are the core concepts of Object Oriented Programming?

Ans:

- The core concepts of Object Oriented Programming (OOP) include:

1. Encapsulation - Encapsulation is the mechanism of wrapping the data (variables) and code (methods) together as a single unit.
 2. Inheritance - Inheritance is the process by which one class acquires the properties and behaviors of another class.
 3. Polymorphism - Polymorphism allows one interface to be used for a general class of actions.
 4. Abstraction - Abstraction is the concept of hiding the implementation details and showing only the essential features of the object.
-
5. Polymorphism - Allows one interface to be used for a general class of actions.
 4. Abstraction - Abstraction is the concept of hiding the implementation details and showing only the essential features of the object.

Features of Object Oriented Programming

- ① Encapsulation - Encapsulation is the mechanism of binding data and methods together within a single unit (class).
- ② Inheritance - Inheritance allows one class to inherit the fields and methods of another class.
- ③ Polymorphism - Polymorphism allows objects to be treated as instances of their parent class, enabling a single interface to represent different underlying forms (data types).
- ④ Abstraction - Abstraction refers to the concept of hiding the implementation details and showing only the essential features of an object.
- ⑤ Reusability - Reusability allows existing code to be used as a base for new programs, saving time and effort.
- ⑥ Security - Security is enhanced through encapsulation, which restricts access to the data within an object and protects it from unauthorized modification.
- ⑦ Flexibility - Flexibility is provided by polymorphism and inheritance, allowing programs to adapt and extend as needed.
- ⑧ Maintainability - Maintainability makes code easier to maintain and modify by using a modular structure that organizes data and functions into classes.

Java OOP Example

Below is a Java example demonstrating object oriented programming concepts like encapsulation, inheritance, and polymorphism:

```
public class Animal {  
    private String name;  
    public Animal(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}  
  
public class Dog extends Animal {  
    public Dog(String name) {  
        super(name);  
    }  
    public void bark() {  
        System.out.println("name + " is barking");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog dog = new Dog("Buddy");  
        dog.bark();  
        System.out.println("Dog name: " + dog.getName());  
    }  
}
```

Output:

- Buddy is barking
- Dog name: Buddy

Constructors in Java

- A constructor in Java is a special type of method that is automatically called when an object is created. Constructors are used to initialize the state of an object.

Types of Constructors

① Default Constructor

- A constructor with no parameters.
- It initializes the object with default values.

② Parameterized Constructor

- A constructor that accepts arguments to initialize the object.
- It allows the setting of initial values for object attributes.

③ No-Argument Constructor

- A constructor that takes no arguments but is explicitly defined by the programmer.
- Similar to the default constructor, it initializes the object with default values.

④ Copy Constructor

- A constructor that creates a new object as a copy of an existing object.
- It is used to duplicate an object by copying the values of its attributes from another object.

Output:

- Buddy is barking
- Dog name: Buddy

@Mastercode.sagar

Method Overloading & Overriding in Java

① Method Overloading :-

- Method overloading is the ability of a class to have multiple methods with the same name but different parameters.
- It is compile time polymorphism (static binding)
- The compiler distinguishes between methods based on the number, type, or order of parameters.

Syntax :

```
returnType methodName (parameterList) { }
```

② Method Overriding :-

- Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its superclass.
- It allows a subclass to modify or extend the behavior of the inherited method
- It is run time polymorphism (dynamic binding).

Rules for Method Overriding :

- ① Method name, return type, and parameters must be same as in superclass.
- ② There must be an IS-A relationship (inheritance).
- ③ Access modifier cannot be more restrictive than superclass.
- ④ Use @Override annotation (optional but recommended).

Example :

Overloading →

```
class Calc {  
    int add(int a, int b) { ... }  
    int sub(int a, int b) { ... }  
}
```

Overriding →

```
class Animal { void sound() { ... } }  
class Dog extends Animal {  
    @Override  
    void sound() { ... }  
}
```


Method Overloading and Overriding in Java

① Method Overloading (Compile Time Polymorphism)

- Method overloading allows multiple methods in the same class to have the same name but different parameters (different number, types or order).
- It is resolved at compile time.

Syntax:

```
returnType methodName(parameter1, parameter2) {  
    // method body  
}
```

Example:

```
public class Calculator {  
    public int add(int a, int b) { return a + b; }  
    public int add(int a, int b, int c) { return a + b + c; }  
    public double add(double a, double b) { return a + b; }  
}
```

② Method Overriding (Run Time Polymorphism)

- Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its superclass.
- The method name, return type and parameters must be the same as in the parent class.
- It is resolved at run time (dynamic method dispatch).

Syntax:

```
class Subclass extends Superclass {  
    @Override  
    returnType methodName(parameters) {  
        // method body  
    }  
}
```

Flow:

↓
Superclass ↔ Subclass
↓
✓ Method is overridden in subclass.

Example:

```
class Animal {  
    void sound() { System.out.println("Animal makes sound"); }  
}  
class Dog extends Animal {  
    @Override  
    void sound() { System.out.println("Dog barks"); }  
}
```


Access Modifiers in Java

- Access modifiers in Java determine the scope of variables, methods, and classes. They control the visibility of these members to other parts of the program.

Access Modifiers in Java

Modifier	Scope	+ Subclass	+ World
Private	Within the same class	×	×
Default	Within the same package	×	×
Protected	Same package + subclasses (even in different packages)	✓	×
Public	From everywhere	✓	✓

✓ = Accessible

×

② Method Overriding :-

Modifier	Scope	+ Subclass	+ World
Private	Within the same class	×	×
Default	Within the same package	×	×
Protected	Same package + subclasses (even in different packages).	✓	×
Public	From everywhere	✓	✓

Access Modifiers in Java

- Access modifiers in Java determine the scope of variables, methods, and classes. They control the visibility of these members to other parts of the program.

Access Modifiers in Java

Modifier	Scope	+ Subclass	+ World
Private	Within the same class	×	×
Default	Within the same package	×	×
Protected	Same package + subclasses (even in different packages)	✓	×
Public	From everywhere	✓	✓

✓ = Accessible

× = Not Accessible

① Encapsulation :-

- Encapsulation is one of the fundamental concepts in object oriented programming.
- It is the technique of wrapping the data (variables) and code (methods) together into a single unit.
- Encapsulation helps to hide the internal state of an object from the outside.
- The internal data of an object can be accessed only through the public methods of the class.

Inheritance in Java

- Inheritance is a mechanism in OOP that allows a class (subclass.) to inherit the properties (fields and methods) of another class (superclass.).
- It represents the IS-A relationship.
- It helps in code reusability and method overriding.

Syntax :

```
class Subclass extends Superclass { }
```

Key Points :

- > The subclass inherits all public and protected members of the superclass.
- > Private members are not inherited.
- >> A class can inherit from only one class (single inheritance in Java).
- >> The subclass can add new fields and methods or override existing ones.

Types of Inheritance :

- ① Single Inheritance - One subclass inherits from one superclass.
- ② Multilevel Inheritance - A class inherits from a class, which in turn inherits from another.
- ③ Hierarchical Inheritance - Multiple subclasses inherit from one superclass.
- ④ Hybrid Inheritance - Combination of two or more types (not supported directly in Java using classes, but possible through

Example :

```
class Animal {  
    void eat(.) { ... }  
}
```

← Superclass.

Animal

(extends)

Inheritance in Java

- Inheritance is a mechanism in OOP that allows a class (child class) to acquire the properties and behaviors (methods) of another class (parent class).
- It represents the IS-A relationship.
- The class whose members are inherited is called superclass (or **base class**).
- The class that inherits is called subclass (or **derived class**).

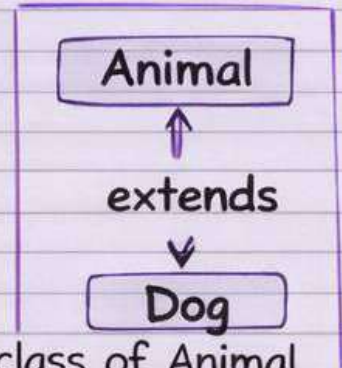
Syntax :

```
class Subclass extends Superclass { }
```

Example :

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}
```

```
class Dog extends Animal { // Dog is a subclass of Animal  
    void bark() {  
        System.out.println("Barking...");  
    }  
}
```



Types of Inheritance in Java :

- Single Inheritance — A class inherits from one class.
- Multilevel Inheritance — A class inherits from another class which is already inherited.
- Hierarchical Inheritance — One class is inherited by multiple classes.

Polymorphism in Java

- Polymorphism is the ability of an object to take many forms, It allows methods to do different things based on the object that is calling them.

Types of Polymorphism :-

1. Compile-Time Polymorphism (Static Binding)

- > Also known as method overloading.
- > It occurs when multiple methods have the same name but differ in the number or type of parameters.
- > It helps in code reusability and method overriding.

Syntax :

```
class Calculator extends Superclass { }
```

Key Points :

- > The subclass inherits all public and protected members of the superclass.
- > Private members are not inherited.
- > A class can inherit from only one class (single inheritance in Java).
- > The subclass can add new fields and methods or override existing ones.

Method Overloading →

```
class Calculator  
    int add(int a, int b) {...}  
    double add(double a,  
               double b) ( ...) {...}  
}
```

Polymorphism Advantages :-

- > Improves code reusability and readability.
- > Allows for a single interface to represent multiple types.
- > Makes it easier to maintain and extend code.
- > Supports method overriding and method overloading.

class Animal →

Animal

Run-Time Polymorphism

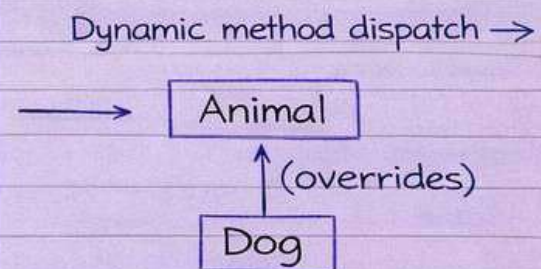
- Run-time polymorphism in Java is achieved through method overriding and is based on dynamic method dispatch.

Method Overriding :-

Key Points :

- > Method overriding occurs in inheritance, when a subclass provides a specific implementation of a method that is already defined in its superclass.
- > The method in the subclass must have the same name, return type, and parameters as the method in the superclass.
- > It helps in achieving runtime polymorphism, allowing a method to be used based on the actual object type that is instantiated.
- > A reference variable of a superclass can refer to an object of a subclass and invoke the overridden method.

```
class Animal {  
    void makeSound() { ... }  
    System.out.println("Animal makes a sound");  
}  
  
class Dog extends Animal {  
    @Override void makeSound() { ... }  
    System.out.println("Dog barks");  
}
```



Output :

- "Dog barks"
- In this example, the reference variable "myDog" of type Animal is assigned to a "Dog" object. When "myDog.makeSound()" is called, it executes the overridden makeSound() method in the "Dog" subclass, outputting 'Dog barks'. This is an example of dynamic method dispatch.